Računalniški Sistemi Homework 1

Andraž Klobučar (63210146)
Filip Gros (id)

1 Introduction

In this report, we analyze the performance characteristics of the O3 out-of-order CPU model using a scaled dot-product attention kernel, a fundamental operation in Transformer architectures. Through a series of simulation experiments, we investigate how parameters,such as reorder buffer size, pipeline width, and register file size affect instruction-level parallelism and overall throughput. We further explore the impact of branch prediction strategies and ROB size in the context of masked attention, a workload with data-dependent control flow typical of autoregressive models.

2 Task 1

2.1 Task 1A

We compare the performance of MinorCPU and O3CPU using key metrics from the same workload.

Metric	MinorCPU	O3CPU
CPI	1.507109	0.885920
Total cycles	100800	59253
IPC	0.663522	1.128770

Table 1: Performance comparison between MinorCPU and O3CPU.

The IPC speedup of O3CPU over MinorCPU is approximately $1.70\times$. This improvement comes from O3CPU's out-of-order execution, which allows it to reorder instructions and avoid stalls. Additionally, speculative execution and a larger reorder buffer enable better utilization of execution units. In contrast, MinorCPU executes instructions strictly in-order, making it more sensitive to pipeline stalls and dependencies.

2.2 Task 1B

We evaluate how ROB size affects IPC for O3CPU with `SEQ_LEN=64`.

ROB Size	16	32	64	128	256
IPC	0.831878	0.992742	1.121239	1.128770	1.128770

Table 2: IPC vs ROB size for O3CPU.

Performance saturates at a ROB size of 128 entries, indicating limited instruction-level parallelism (ILP). The workload is dominated by a dependency chain in the dot-product loop,

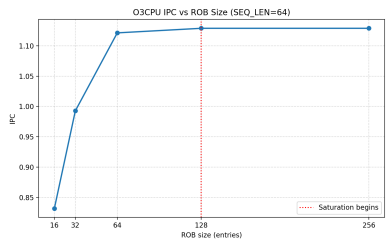

Figure 1: IPC vs ROB size at SEQ_LEN = 64.

where each iteration depends on the previous result. With a long critical path (384 cycles), the processor quickly exhausts independent instructions. Increasing the ROB beyond 128 entries therefore provides no benefit, as the limitation is algorithmic rather than architectural.

2.3 Task 1C

We compare the original and optimized attention kernel while varying pipeline width.

Pipeline Width	2	4	8
Original	1.128770	1.153991	1.146907
Optimized	1.388050	1.525176	1.643418

Table 3: IPC vs pipeline width for original and optimized kernels.

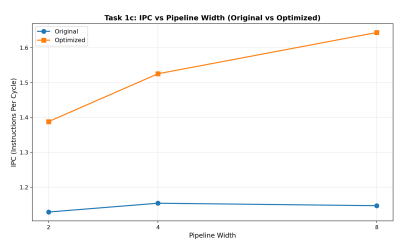

Figure 2: IPC vs pipeline width.

The optimized version achieves higher IPC and scales better with pipeline width because it reduces dependency chains using independent accumulators. As width increases, the IPC gap widens, showing that only the optimized version can effectively utilize additional execution resources. The original

version remains limited by serialization, so wider pipelines provide little benefit. This demonstrates that sufficient ILP is required to take advantage of wider microarchitectures.

2.4 Task 1D

We analyze the effect of register file size (64, 96, 128) on IPC and stalls for both kernel versions.

	64	96	128
Original IPC	1.120506	1.244335	1.230802
Original Stalls	17338	9583	145
Optimized IPC	1.377836	1.616652	1.549267
Optimized Stalls	2750	27	131

Table 4: IPC and register stalls vs register file size.

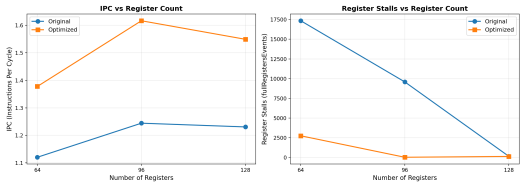

Figure 3: IPC and stalls vs register file size.

IPC improves from 64 to 96 registers for both versions due to reduced register stalls, but decreases at 128 despite stalls being nearly eliminated. This indicates that performance saturates at 96 registers. Beyond this point, the workload is limited by instruction dependencies rather than register pressure, and additional registers introduce overhead without improving performance.

3 Task 2

In this task, we investigate the impact of branch prediction on the performance of a masked attention kernel. The workload applies a causal mask to prevent each token from attending to future positions, introducing data-dependent branches whose behavior changes systematically across the sequence. This makes the workload particularly interesting for analyzing branch prediction strategies and their effects on the processors performance.

3.1 Task 2A

In this subtask, we compiled and ran the masked attention workload using the baseline O3CPU configuration. We recorded key performance metrics such as the total number of instructions committed, total cycles, IPC, the number of branch instructions, and branch mispredictions. These baseline results are displayed in Table 5.

3.2 Task 2B

In this subtask, we evaluated the effectiveness of several branch predictors: TournamentBP, LocalBP, BiModeBP, and TAGE. We evaluated these by running the masked attention workload with each predictor while keeping all other parameters fixed. For each predictor, we recorded the number of branch mispredictions and the resulting IPC. The recorded metrics are displayed in Table 6.

Metric	Value
Total instructions committed	2,194,513
Total cycles	1,774,074
IPC	1.236991
Branch instructions committed	335,874
Branch mispredictions	501

Table 5: Baseline performance and branch statistics for masked attention workload on O3CPU.

The TAGE branch predictor achieved the lowest misprediction count at 499. This is because TAGE uses multiple predictor tables with different history lengths, which allows it to capture both short- and long-term branch correlations. This makes it very efficient for workloads with complex branch patterns. The other branch predictors rely on shorter branch history patterns, which makes them less reliable and more prone to mispredicting branches.

Predictor	Branch mispredictions	IPC
TAGE	499	1.237033
LocalBP	5,430	1.203245
Tournament	4,673	1.214289
BimodeBP	4,788	1.212354

Table 6: Comparison of branch predictors for masked attention workload: mispredictions and IPC.

3.3 Task 2C

In this subtask, we used the best-performing branch predictor from the previous experiment and sweeped the Reorder Buffer (ROB) size over several values. For each ROB size, we measured the number of branch mispredictions, IPC, and the number of squashed instructions due to mispredictions. This analysis reveals how increasing the ROB size affects both processor throughput and the penalty incurred from branch mispredictions. The recorded metrics are displayed in Table 7.

We can observe from the metrics that as the ROB size increases, the IPC also increases, indicating better processor throughput. Meanwhile, the number of flushed instructions per misprediction decreases. A larger ROB allows the processor to keep more instructions in progress, including independent instructions that are not dependent on unresolved branches. This means the processor can commit more correct-path instructions before a misprediction is detected, so when a misprediction does occur, there are fewer speculative instructions left to be discarded. Thus, a larger ROB both improves performance and reduces the penalty for mispredictions.

ROB Size	Misprediction count	IPC	Squashed instruction count
32	499	1.088510	520
64	499	1.238335	478
128	499	1.237033	442

Table 7: Effect of ROB size on branch mispredictions, IPC, and squashed instructions using TAGE predictor.

4 Conclusion

Task 1 shows that O3CPU outperforms MinorCPU due to out-of-order execution, but performance is ultimately limited by inherent data dependencies in the workload. It also demonstrates that improving instruction-level parallelism is essential to effectively utilize wider pipelines and larger hardware resources. In the branch experiments, the TAGE predictor achieved the lowest misprediction rate, effectively handling the complex, shifting branch patterns in masked attention computation.